Travaux Pratiques 2 : Conteneurisation d'une application Web

Institut Supérieur d'Informatique
Département Génie des Télécommunications et Réseaux (GTR)
Module : Cloud Computing & Virtualisation
Groupes : M1 SSII
Enseignant : Safa Réjichi
Mail : talel.chaanbi@etudiant-isi.utm.tn
Réalisé par : Talel Chaanbi

Objectifs

- Création d'une stack LAMP au moyen de Docker.
- Manipulation de Dockerfile.
- Manipulation de volumes.
- Appréhension de la notion de réseau avec Docker.

Partie 1 : Manipulation de Dockerfile

I. Préparation du répertoire de travail

1. Ouvrez la machine virtuelle et créez un répertoire de travail.
2. Désarchivez le contenu fourni, avec l'arborescence suivante :

```
sources/
├── app/
│   ├── db-config.php
│   ├── index.php
│   └── validation.php
└── db/
    └── articles.sql
```

3. Créez un fichier `Dockerfile` à la racine du projet.

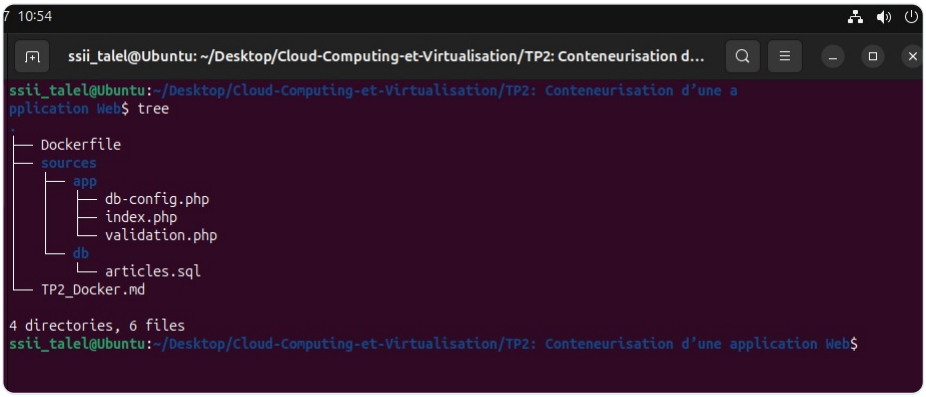


Figure 1 : Arborescence du projet TP2 (sources/app, sources/db, Dockerfile)

II. Création de l'image LAMP

1. Dockerfile utilisé

```
# --------------- DÉBUT COUCHE OS -------------------
FROM ubuntu:latest
# --------------- FIN COUCHE OS ---------------------

# MÉTADONNÉES DE L'IMAGE
LABEL version="1.0" maintainer="CHAANBI Talel <talel.chaanbi@etudiant-isi.utm.tn>"

# VARIABLES TEMPORAIRES
ARG APT_FLAGS="-q -y"
ARG DOCUMENTROOT="/var/www/html"

# --------------- DÉBUT COUCHE APACHE ---------------
RUN apt-get update -y && DEBIAN_FRONTEND=noninteractive apt-get install ${APT_FLAGS}
apache2
# --------------- FIN COUCHE APACHE -----------------

# --------------- DÉBUT COUCHE MYSQL ----------------
RUN apt-get install ${APT_FLAGS} mariadb-server
COPY sources/db/articles.sql /
# --------------- FIN COUCHE MYSQL ------------------

# --------------- DÉBUT COUCHE PHP ------------------
RUN DEBIAN_FRONTEND=noninteractive apt-get install ${APT_FLAGS} \
    php-mysql \
    php && \
    rm -f ${DOCUMENTROOT}/index.html && \
    apt-get autoclean -y

COPY sources/app ${DOCUMENTROOT}
# --------------- FIN COUCHE PHP --------------------

# OUVERTURE DU PORT HTTP
EXPOSE 80

# RÉPERTOIRE DE TRAVAIL
WORKDIR ${DOCUMENTROOT}

# DÉMARRAGE DES SERVICES LORS DE L'EXÉCUTION DE L'IMAGE
ENTRYPOINT service mariadb start && mysql < /articles.sql && apache2ctl -D FOREGROUND
```

Expliquez le contenu du Dockerfile :

> ✏ **Réponse :**
>
> - `FROM ubuntu:latest` : image de base Ubuntu.
> - Installation d'Apache (`apache2`) pour le serveur Web.
> - Installation de MariaDB pour la base de données.
> - Copie du script SQL `articles.sql`.
> - Installation de PHP + extension `php-mysql`.
> - Copie de l'application PHP vers `/var/www/html`.
> - Exposition du port `80`.
> - Démarrage MariaDB + import SQL + lancement Apache au démarrage du conteneur.

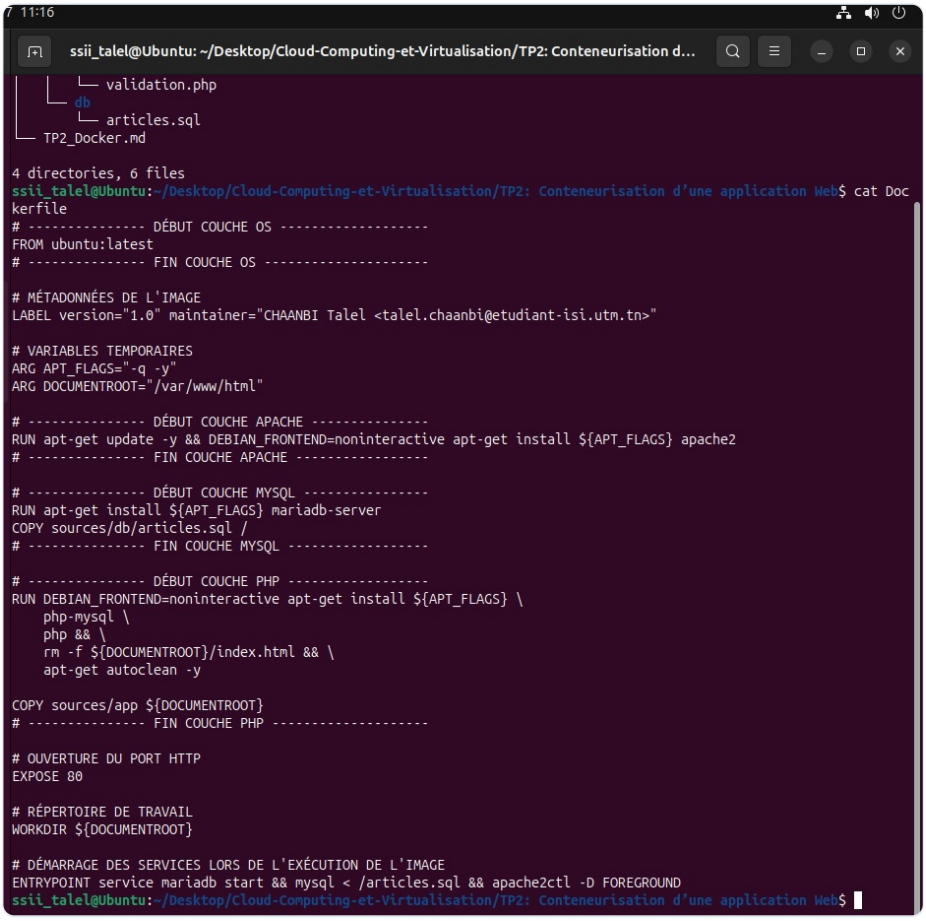


Figure 2 : Contenu du Dockerfile de la stack LAMP

---

2. Build de l'image

```
docker build -t my_lamp .
```

```
docker images
```

Figure 3 : Construction de l'image `my_lamp` et vérification via `docker images`

III. Création et exécution du conteneur

1. Lancer le conteneur :

```
docker run -d --name my_lamp_c -p 8080:80 my_lamp
```

2. **Expliquez l'option `-p 8080:80` :**

> ✏ **Réponse :** le port `8080` de la machine hôte est redirigé vers le port `80` du conteneur (serveur Apache).

3. Donner les droits nécessaires :

```
docker exec my_lamp_c chmod 777 /var/www/html/index.php
docker exec my_lamp_c chmod 777 /var/www/html/db-config.php
docker exec my_lamp_c chmod 777 /var/www/html/validation.php
```

4. Vérifier l'état du conteneur :

```
docker ps
```

5. En cas de problème, consulter les logs :

```
docker logs -ft my_lamp_c
```

6. Tester l'application :

- URL : `http://localhost:8080/`

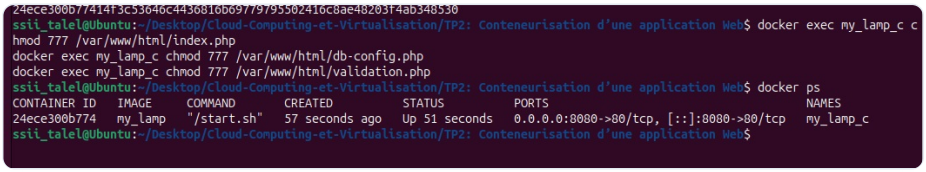


Figure 4 : Lancement du conteneur `my_lamp_c` et vérification du statut

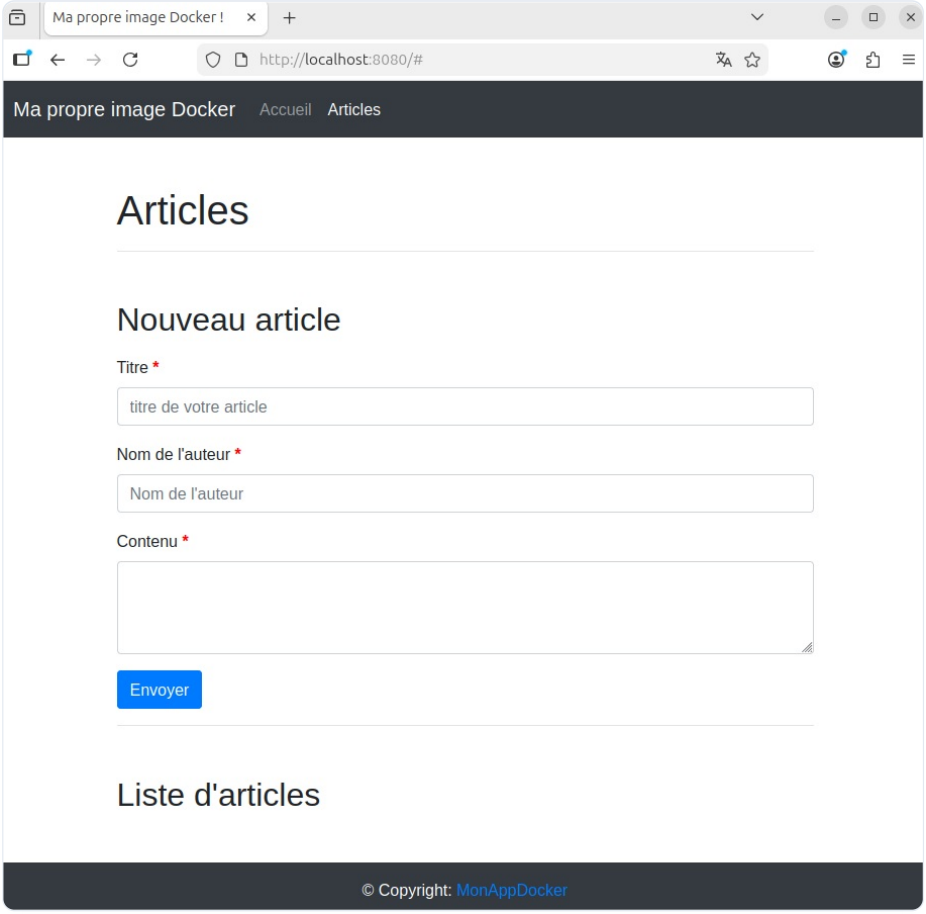


Figure 5 : Affichage de l'application sur `http://localhost:8080/`

IV. Test de persistance sans volume

1. Ajouter un nouvel article depuis l'interface.

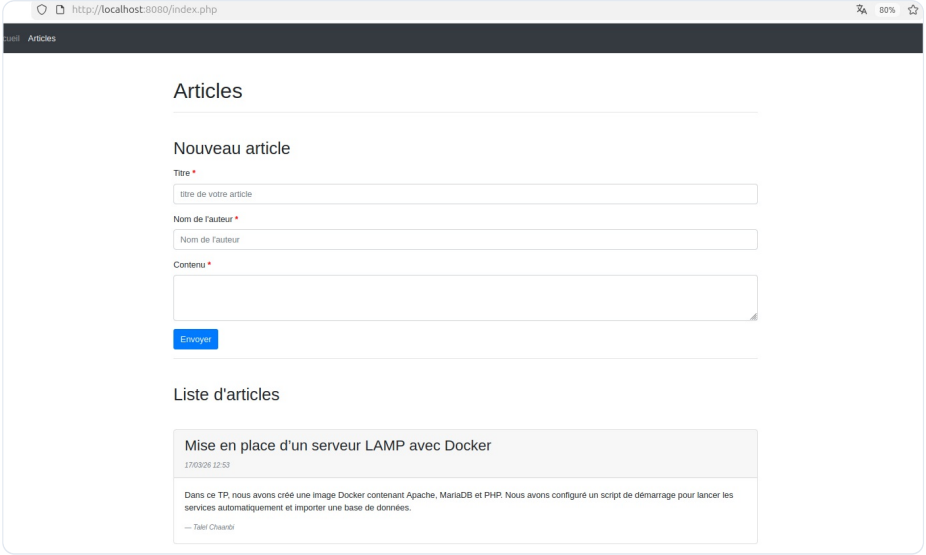


Figure 6 : Ajout d'un nouvel article via l'interface

2. Détruire puis recréer le conteneur :

```
docker stop my_lamp_c
docker rm my_lamp_c
docker run -d --name my_lamp_c -p 8080:80 my_lamp
docker exec my_lamp_c chmod 777 -R /var/www/html
```

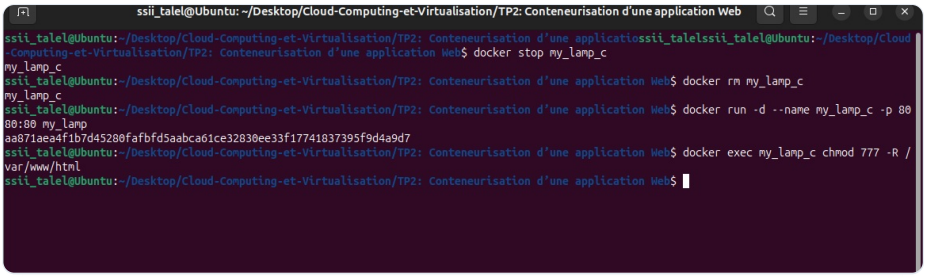


Figure 7 : Suppression et recréation du conteneur (exécution des commandes)

3. Vérifier l'existence de l'article déjà ajouté.

Expliquez le résultat :

> ✏ **Réponse :** l'article est perdu, car les données de la base étaient stockées dans le conteneur supprimé (pas de volume persistant).

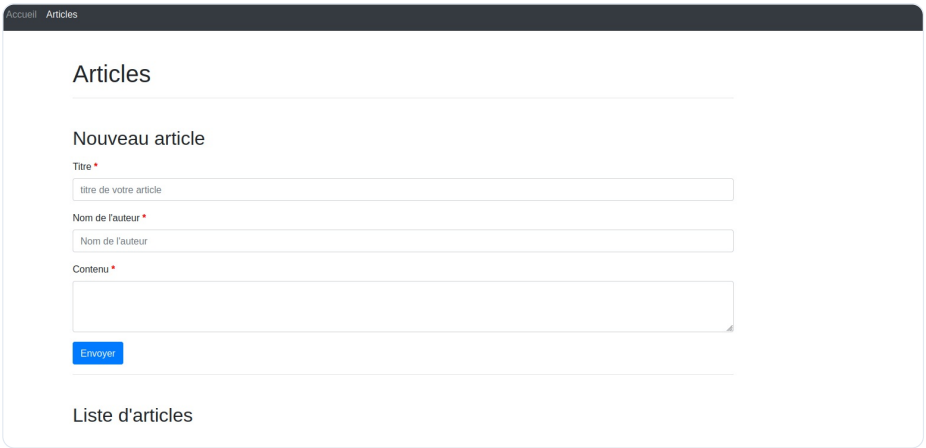


Figure 8 : Perte des données après suppression/recréation du conteneur sans volume (Liste vide)

Partie 2 : Manipulation de Volume

I. Création et utilisation d'un volume Docker

1. Créer le volume :

```
docker volume create --name mysqldata
```

2. Lister les volumes :

```
docker volume ls
```

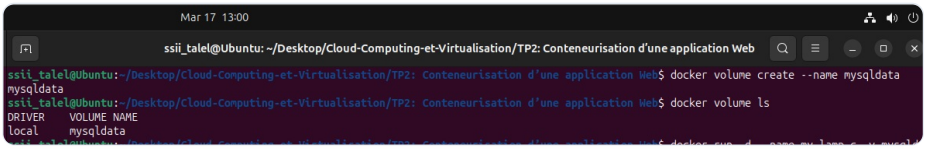


Figure 9 : Liste des volumes Docker (`docker volume ls`)

3. Exécuter le conteneur avec le volume :

```
docker run -d --name my_lamp_c -v mysqldata:/var/lib/mysql -p 8080:80 my_lamp
```

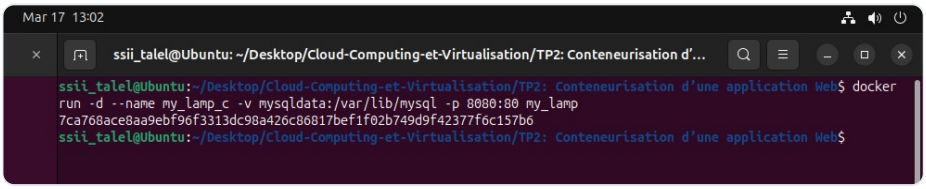


Figure 10 : Exécution du conteneur avec le volume `mysqldata` monté

4. **Expliquez l'option `-v mysqldata:/var/lib/mysql` :**

> ✏ **Réponse :** le volume Docker `mysqldata` est monté dans `/var/lib/mysql` du conteneur, ce qui rend les données MariaDB persistantes.

II. Vérification de la persistance avec volume

1. Ajouter un article via `http://localhost:8080/`.

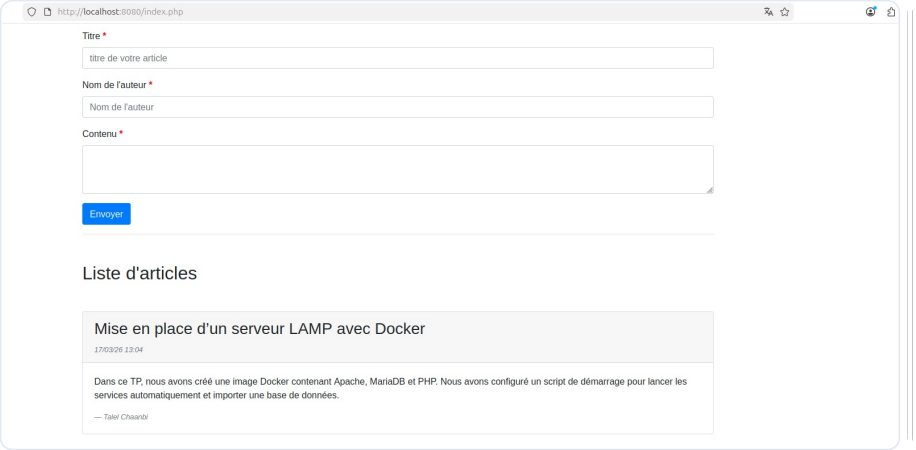


Figure 11 : Ajout d'un article via l'application (avec volume monté)

2. Détruire puis recréer le conteneur avec le même volume :

```
docker stop my_lamp_c
docker rm my_lamp_c
docker run -d --name my_lamp_c -v mysqldata:/var/lib/mysql -p 8080:80 my_lamp
```

3. Vérifier l'existence de l'article ajouté.

Expliquez le résultat :

> ✏ **Réponse :** l'article est conservé, car la base de données est stockée dans le volume `mysqldata`, indépendant du cycle de vie du conteneur.

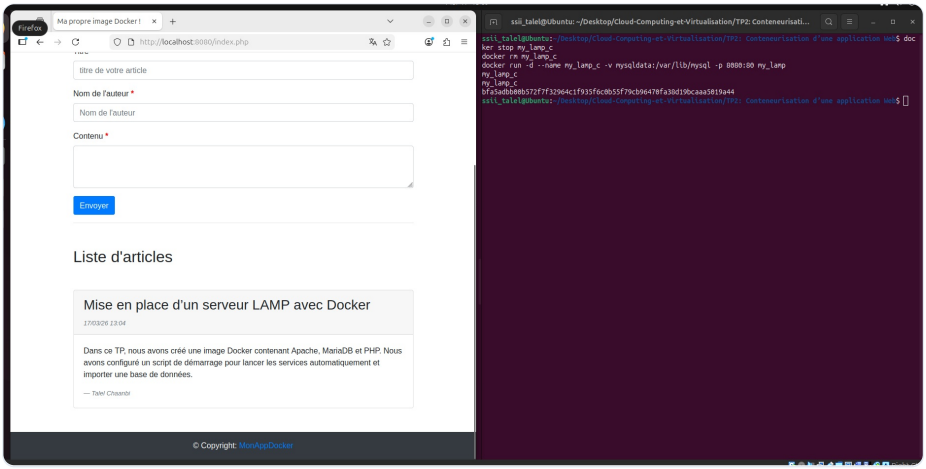


Figure 12 : Données conservées après recréation du conteneur grâce au volume

---

Partie 3 : Manipulation du Réseau

I. Vérification du réseau Docker

1. Lister les réseaux Docker :

```
docker network ls
```

```
docker rm my_lamp_c
docker run -d --name my_lamp_c -v mysqldata:/var/lib/mysql -p 8080:80 my_lamp
my_lamp_c
my_lamp_c
bfa5adbb00b572f7f32964c1f935f6c0b55f79cb96470fa38d19bcaaa5019a44
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docke
r network ls
NETWORK ID     NAME      DRIVER    SCOPE
e537c886dde5   bridge    bridge    local
f1c6a3652639   host      host      local
dcfb0f9ae895   none      null      local
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docke
r exec my_lamp_c ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

Figure 13 : Liste des réseaux Docker (`docker network ls`)

2. Donner l'adresse IP de la machine hôte.

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:d5:17 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 72899sec preferred_lft 72899sec
    inet6 fd17:625c:f037:2:ce00:ac5c:2d6c:f315/64 scope global temporary dynamic
       valid_lft 86052sec preferred_lft 14052sec
    inet6 fd17:625c:f037:2:a00:27ff:fefe:d517/64 scope global dynamic mngtmpaddr
       valid_lft 86052sec preferred_lft 14052sec
    inet6 fe80::a00:27ff:fefe:d517/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether d2:74:4d:7b:27:75 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

Figure 14 : Adresse IP de la machine hôte (extrait)

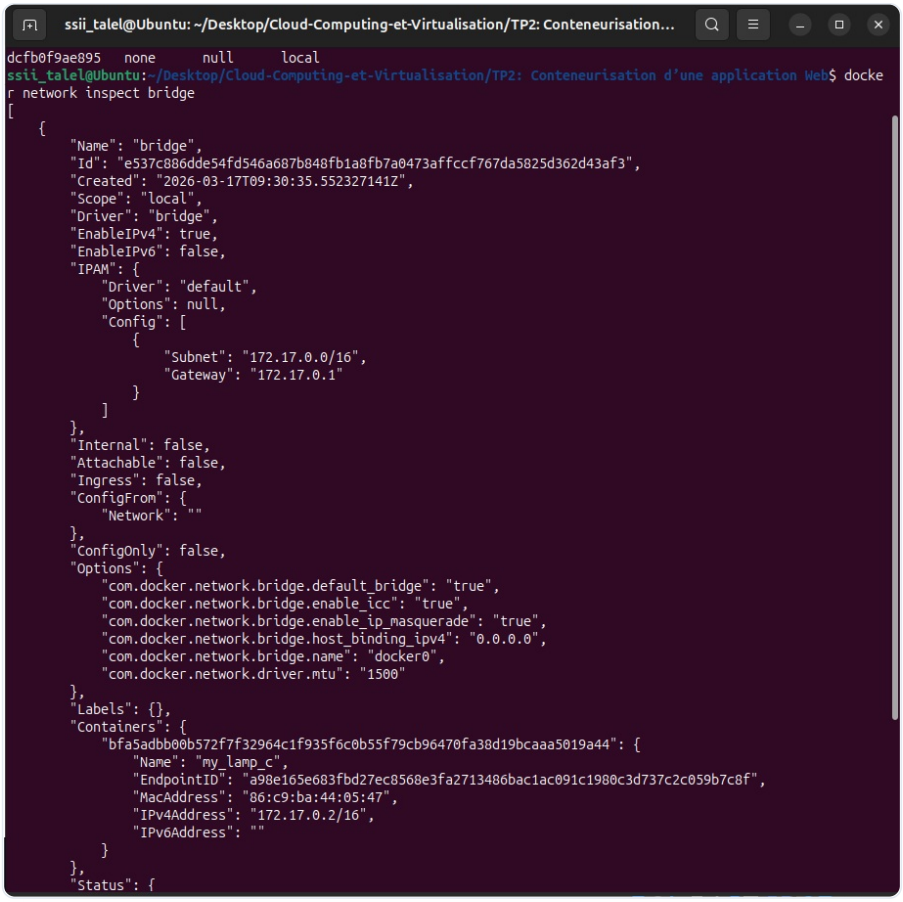
3. Afficher l'adresse IP du conteneur :

```
docker exec my_lamp_c ip add
```

```
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docke
r exec my_lamp_c ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if20: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:c9:ba:44:05:47 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ ip ad
```

Figure 15 : Adresse IP du conteneur `my_lamp_c` sur le réseau bridge

4. Vérifier qu'ils sont sur le même réseau bridge.



```
ssii_talel@Ubuntu: ~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation...
dcfb0f9ae895   none      null      local
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docke
r network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "e537c886dde54fd546a687b848fb1a8fb7a0473affccf767da5825d362d43af3",
        "Created": "2026-03-17T09:30:35.552327141Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.bridge.default_bridge": "true",
            "com.docker.network.bridge.enable_icc": "true",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
            "com.docker.network.bridge.name": "docker0",
            "com.docker.network.driver.mtu": "1500"
        },
        "Labels": {},
        "Containers": {
            "bfa5adbb00b572f7f32964c1f935f6c0b55f79cb96470fa38d19bcaaa5019a44": {
                "Name": "my_lamp_c",
                "EndpointID": "a98e165e683fbd27ec8568e3fa2713486bac1ac091c1980c3d737c2c059b7c8f",
                "MacAddress": "86:c9:ba:44:05:47",
                "IPv4Address": "172.17.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
```

Figure 16 : Vérification du réseau bridge et correspondance des subnets > ✏ **Réponse :** le conteneur est attaché au réseau bridge Docker (interface `docker0` côté hôte) et reçoit une IP privée de ce sous-réseau.

Partie 4 : Publier son image dans Docker Hub

I. Connexion et préparation

1. Créer un repository public sur `https://hub.docker.com/`.
2. Se connecter depuis le terminal :

```
docker login
```

```
ssii_talel@Ubuntu: ~
ssii_talel@Ubuntu:~$ docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: FFTX-JTMH
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser…

WARNING! Your credentials are stored unencrypted in '/home/ssii_talel/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ssii_talel@Ubuntu:~$
```

Figure 17 : Connexion au compte Docker Hub (`docker login`)

3. Vérifier la présence de l'image locale :

```
docker images
```

```
ssii_talel@Ubuntu:~$ docker images
                                                                    i Info →   U   In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
image_m1ssii:latest    c89ee50e85d8        226MB         73.2MB
my_lamp:latest         22f5be545aef        792MB          169MB    U
ubuntu:latest          d1e2e92c075e        119MB         31.7MB    U
ssii_talel@Ubuntu:~$
```

Figure 18 : Vérification de la présence de l'image locale (`docker images`)

4. Tagger l'image :

```
docker tag my_lamp talelchaanbi/my_lamp:first
```

5. Vérifier le tag :

```
docker images
```

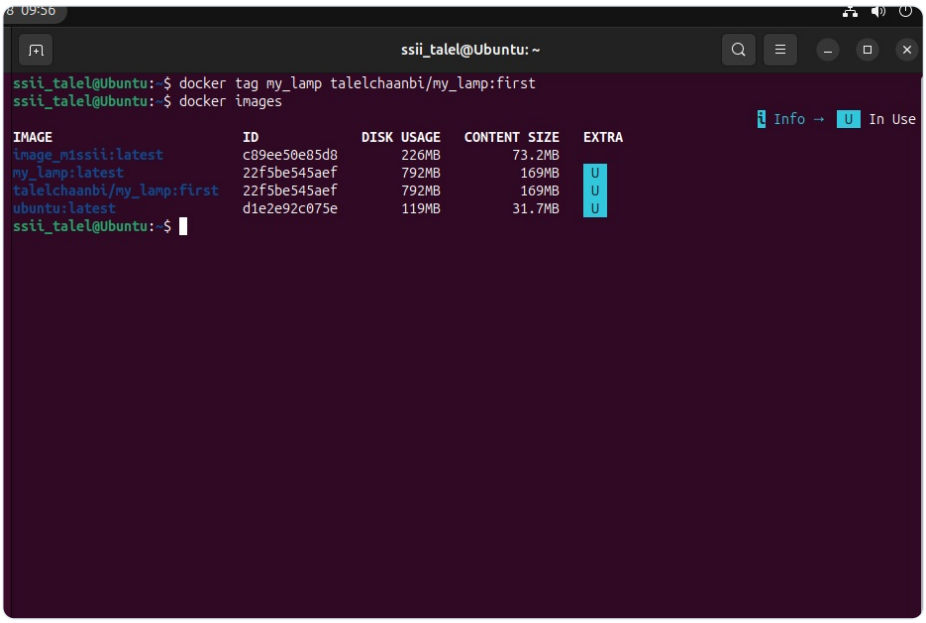


Figure 19 : Tag de l'image et vérification dans la liste des images

---

II. Publication

```
docker push talelchaanbi/my_lamp:first
```

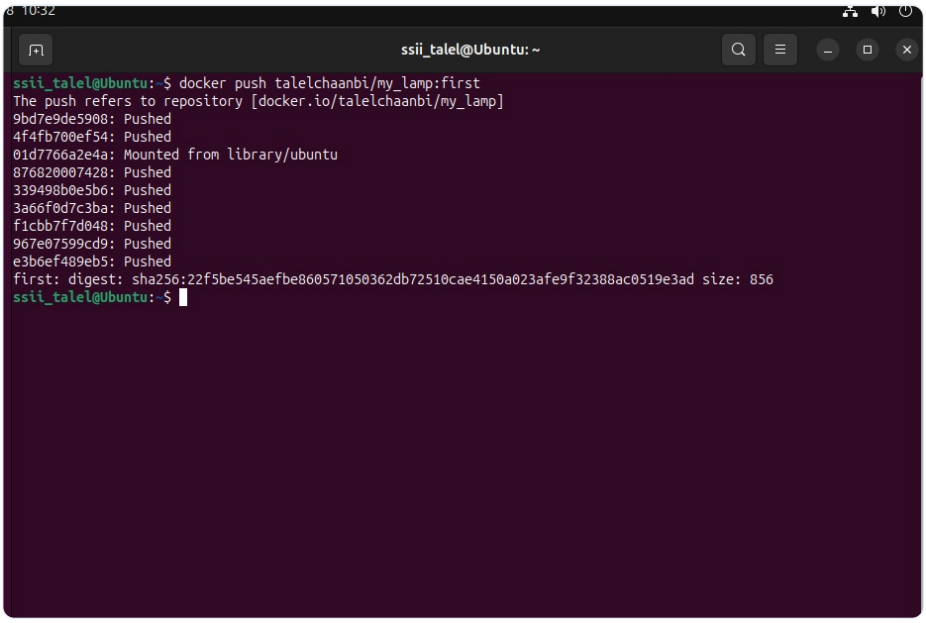


Figure 20 : Exécution de la commande `docker push` vers Docker Hub

Vérification :

> ✏ **Réponse :** l'image est visible dans le repository Docker Hub après le `push`.

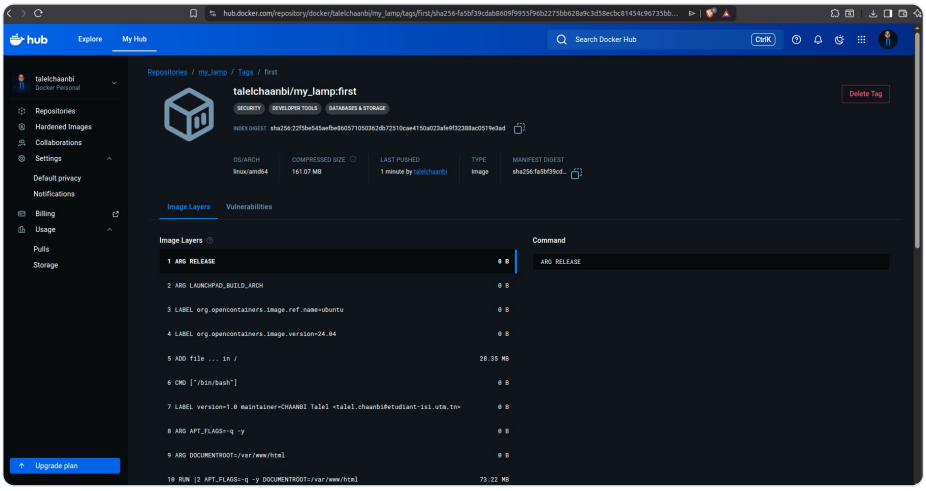


Figure 21 : Image `my_lamp` publiée et visible sur Docker Hub

Bonnes pratiques de sécurité (TP2)

1. Éviter de travailler en root permanent.
2. Limiter l'usage du groupe `docker` (droits équivalents root).
3. Ne pas utiliser `--privileged` sans besoin réel.
4. Exposer uniquement les ports nécessaires.
5. Utiliser des volumes pour les données critiques.
6. Vérifier les images avant usage et les mettre à jour.
7. Sauvegarder régulièrement les volumes et journaliser les actions.

Fin du TP 2 — Conteneurisation d'une application Web

Modifications apportées au TP

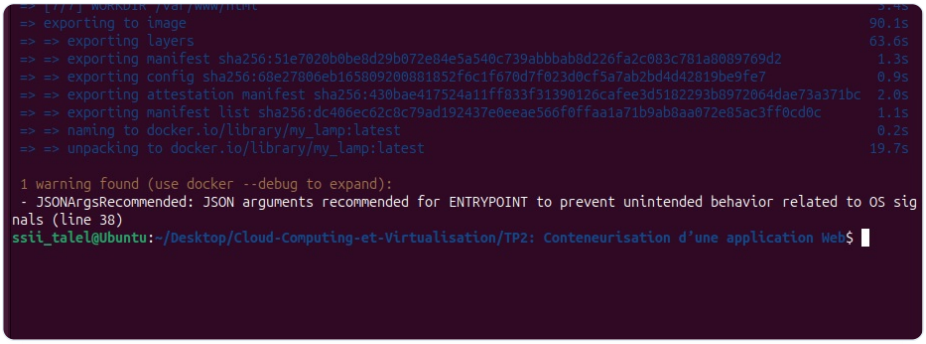


Figure 22 : Capture d'écran montrant le problème détecté lors du démarrage (import SQL échoué)

Pour améliorer la robustesse et la maintenabilité de la stack LAMP fournie, nous avons apporté les modifications suivantes :

- Ajout du script de démarrage `start.sh` (présent à la racine du TP). Ce script : démarre MariaDB, importe la base `articles.sql` une seule fois (repère via le fichier `/var/lib/mysql/.db_initialized`) puis lance Apache en avant-plan.

- Mise à jour du `Dockerfile` pour copier `start.sh` dans l'image et utiliser `ENTRYPOINT ["/start.sh"]` au lieu d'une commande inline. Cela permet une meilleure gestion des signaux et un démarrage plus clair des services.

Motifs : fiabilité du démarrage (éviter les erreurs d'import SQL causées par MariaDB non initialisé), éviter un ré-import à chaque redémarrage, et respecter les bonnes pratiques Docker (script de démarrage exécutable, `ENTRYPOINT` en forme JSON).

Captures :

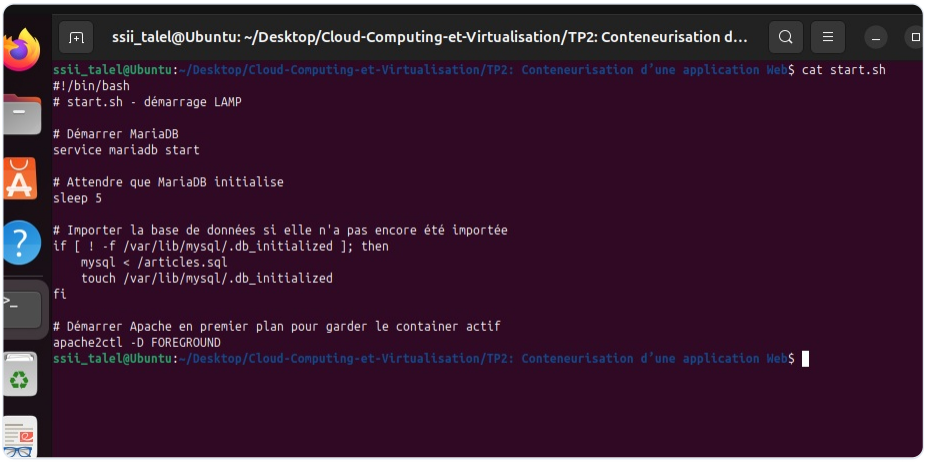


Figure 23 : Contenu du script `start.sh` ajouté

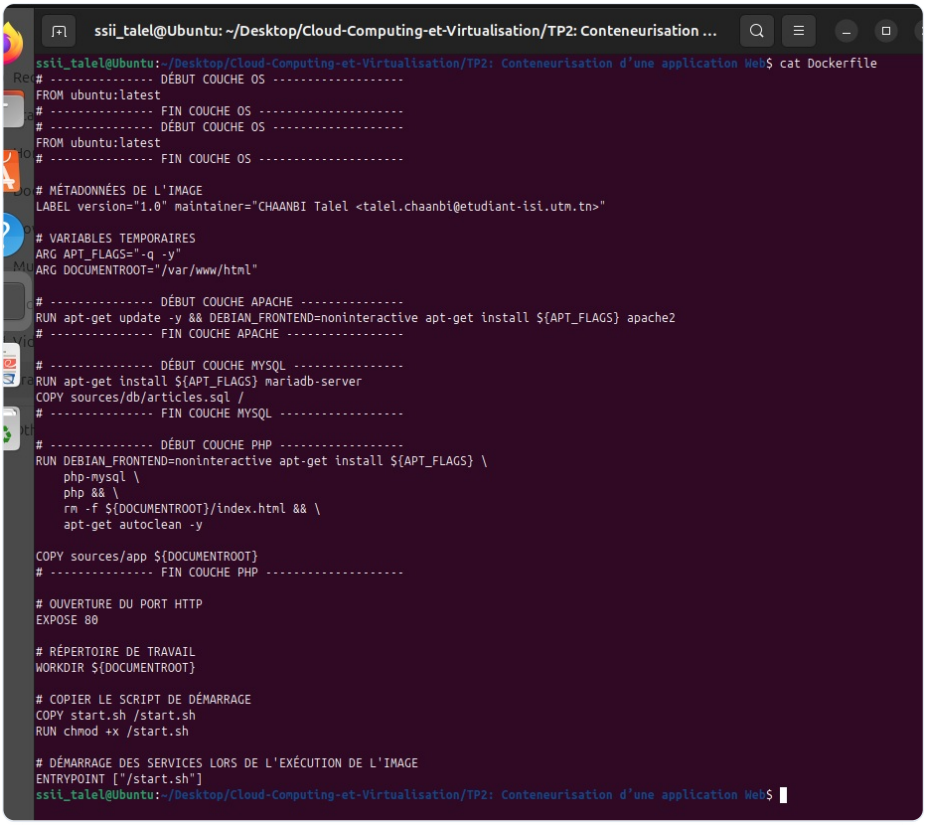


Figure 24 : Extrait du `Dockerfile` montrant la copie et l'ENTRYPOINT

```
ssii_talel@Ubuntu: ~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d...
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docker
images
                                                                                                  i Info →   U  In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
image_m1ssii:latest    c89ee50e85d8        226MB         73.2MB
ubuntu:latest          d1e2e92c075e        119MB         31.7MB    U
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$ docker
build -t my_lamp .
[+] Building 62.6s (14/14) FINISHED                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                          0.8s
 => => transferring dockerfile: 1.44kB                                                                        0.1s
 => [internal] load metadata for docker.io/library/ubuntu:latest                                              1.6s
 => [internal] load .dockerignore                                                                             0.6s
 => => transferring context: 2B                                                                               0.0s
 => [stage-1 1/9] FROM docker.io/library/ubuntu:latest@sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203  1.2s
 => => resolve docker.io/library/ubuntu:latest@sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c  1.1s
 => [internal] load build context                                                                             0.7s
 => => transferring context: 713B                                                                             0.1s
 => CACHED [stage-1 2/9] RUN apt-get update -y && DEBIAN_FRONTEND=noninteractive apt-get install -q -y apac  0.0s
 => CACHED [stage-1 3/9] RUN apt-get install -q -y mariadb-server                                             0.0s
 => CACHED [stage-1 4/9] COPY sources/db/articles.sql /                                                       0.0s
 => CACHED [stage-1 5/9] RUN DEBIAN_FRONTEND=noninteractive apt-get install -q -y     php-mysql     php &&   0.0s
 => CACHED [stage-1 6/9] COPY sources/app /var/www/html                                                       0.0s
 => CACHED [stage-1 7/9] WORKDIR /var/www/html                                                                0.0s
 => [stage-1 8/9] COPY start.sh /start.sh                                                                     2.2s
 => [stage-1 9/9] RUN chmod +x /start.sh                                                                      7.8s
 => exporting to image                                                                                       39.9s
 => => exporting layers                                                                                       9.8s
 => => exporting manifest sha256:fa5bf39cdab8609f9955f96b2275bb628a9c3d58ecbc81454c96735bb2a9cf52             1.0s
 => => exporting config sha256:c7cfc894c43b0f31b9ee80f23bc4c4bd1c498361dd02f01ec88e3c3f32c8ee7d               1.0s
 => => exporting attestation manifest sha256:2ac10fa7dfed0baa50c19a23a908eb52d1143161038cf242ae7f88b5c4c47b  1.7s
 => => exporting manifest list sha256:22f5be545aefbe860571050362db72510cae4150a023afe9f32388ac0519e3ad       0.9s
 => => naming to docker.io/library/my_lamp:latest                                                             0.2s
 => => unpacking to docker.io/library/my_lamp:latest                                                         23.7s
ssii_talel@Ubuntu:~/Desktop/Cloud-Computing-et-Virtualisation/TP2: Conteneurisation d'une application Web$
```

Figure 25 : Exemple de sortie de `docker build` / `docker run` montrant l'image `my_lamp` démarrée